

🏀 NBA Player Longevity Prediction — Gaussian Naive Bayes

Naive Bayes Model Evaluation | Sports Analytics Project

Goal: Predict whether an NBA player will last 5+ years in the league (target_5yrs)

Classifier: Gaussian Naive Bayes (Scikit-learn)

Audience: Data Science team + Scouting department

```
# — Imports —————
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
import seaborn as sns

from sklearn.naive_bayes import GaussianNB
from sklearn.model_selection import train_test_split
from sklearn.metrics import (
    confusion_matrix, ConfusionMatrixDisplay,
    precision_score, recall_score, f1_score,
    classification_report, roc_auc_score, roc_curve
)
from sklearn.preprocessing import StandardScaler
from sklearn.inspection import permutation_importance

import warnings
warnings.filterwarnings("ignore")

# Consistent aesthetics
plt.rcParams.update({
    "figure.dpi": 130,
    "axes.spines.top": False,
    "axes.spines.right": False,
    "font.family": "DejaVu Sans",
})
PALETTE = {"won": "#1d428a", "lost": "#c8102e", "neutral": "#adb5bd"}
print("✅ Libraries loaded successfully.")
```

✅ Libraries loaded successfully.

1 · Load & Inspect the Dataset

We load the **engineered NBA player dataset** and confirm target_5yrs is the binary classification target.

- 0 → Player did NOT last 5 years in the NBA ("bust")
- 1 → Player DID last 5+ years ("durable")

```
df = pd.read_csv("extracted_nba_players_data.csv")

print(f"Dataset shape: {df.shape[0]:,} rows × {df.shape[1]} columns")
print("\n— Column overview —————")
print(df.dtypes.to_string())
print("\n— Missing values —————")
```

```
print(df.isnull().sum().to_string())
print("\n— First 5 rows —")
df.head()
```

Dataset shape: 1,340 rows × 11 columns

— Column overview —

```
fg          float64
3p          float64
ft          float64
reb         float64
ast         float64
stl         float64
blk         float64
tov         float64
target_5yrs int64
total_points float64
efficiency  float64
```

— Missing values —

```
fg          0
3p          0
ft          0
reb         0
ast         0
stl         0
blk         0
tov         0
target_5yrs 0
total_points 0
efficiency  0
```

— First 5 rows —

	fg	3p	ft	reb	ast	stl	blk	tov	target_5yrs	total_points	efficiency
0	34.7	25.0	69.9	4.1	1.9	0.4	0.4	1.3	0	266.4	0.270073
1	29.6	23.5	76.5	2.4	3.7	1.1	0.5	1.6	0	252.0	0.267658
2	42.2	24.4	67.0	2.2	1.0	0.5	0.3	1.0	0	384.8	0.339869
3	42.6	22.6	68.9	1.9	0.8	0.6	0.1	1.0	1	330.6	0.491379
4	52.4	0.0	67.4	2.5	0.3	0.3	0.4	0.8	1	216.0	0.391304



Next steps: [Generate code with df](#) [New interactive sheet](#)

```
# Target variable distribution
counts = df["target_5yrs"].value_counts()
pct     = df["target_5yrs"].value_counts(normalize=True) * 100

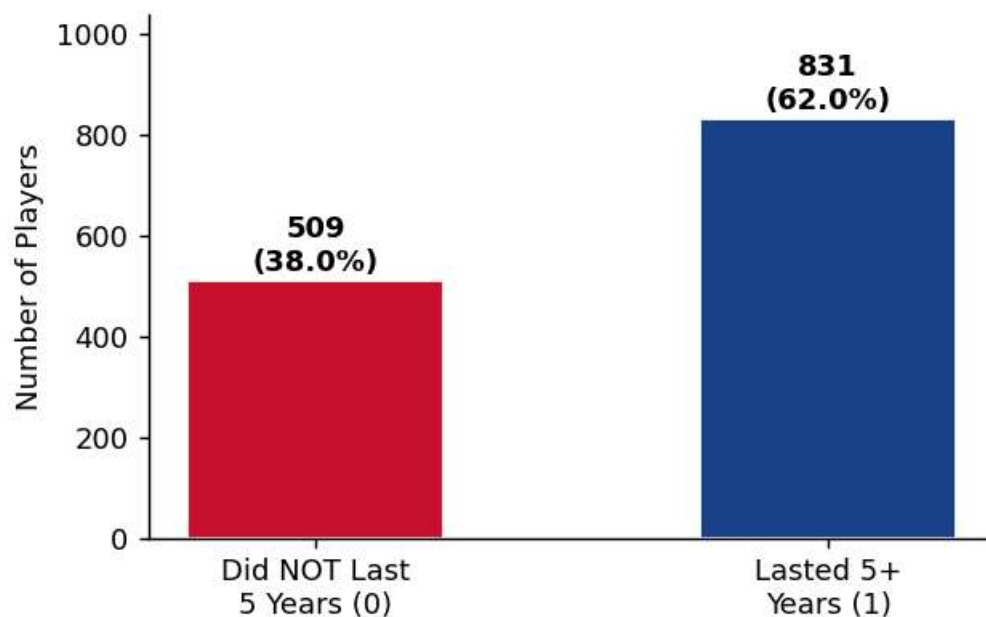
print("\n— Target Distribution (target_5yrs) —")
print(f"  Lasted 5+ years  (1): {counts[1]:>4}  ({pct[1]:.1f}%)"
print(f"  Did NOT last 5yr (0): {counts[0]:>4}  ({pct[0]:.1f}%)"
print(f"  Total:                {len(df):>4}")

fig, ax = plt.subplots(figsize=(5, 3.5))
bars = ax.bar(["Did NOT Last\n5 Years (0)", "Lasted 5+\nYears (1)"],
              [counts[0], counts[1]],
              color=[PALETTE["lost"], PALETTE["won"]], width=0.5, edgecolor="white")
for bar, val, pct_val in zip(bars, [counts[0], counts[1]], [pct[0], pct[1]]):
    ax.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 8,
            f"{val}\n({pct_val:.1f}%)", ha="center", va="bottom", fontsize=10, fontweight="bold")
ax.set_title("NBA Player Longevity – Target Distribution", fontsize=12, fontweight="bold", pad=12)
ax.set_ylabel("Number of Players")
ax.set_ylim(0, max(counts) * 1.25)
```

```
plt.tight_layout()
plt.savefig("fig_01_target_distribution.png", bbox_inches="tight")
plt.show()
```

— Target Distribution (target_5yrs) —————
Lasted 5+ years (1): 831 (62.0%)
Did NOT last 5yr (0): 509 (38.0%)
Total: 1340

NBA Player Longevity — Target Distribution



✓ 2 · Exploratory Data Analysis (EDA)

Before modeling, we examine how each feature distributes across the two target classes. This helps us develop intuition about which stats might drive longevity predictions.

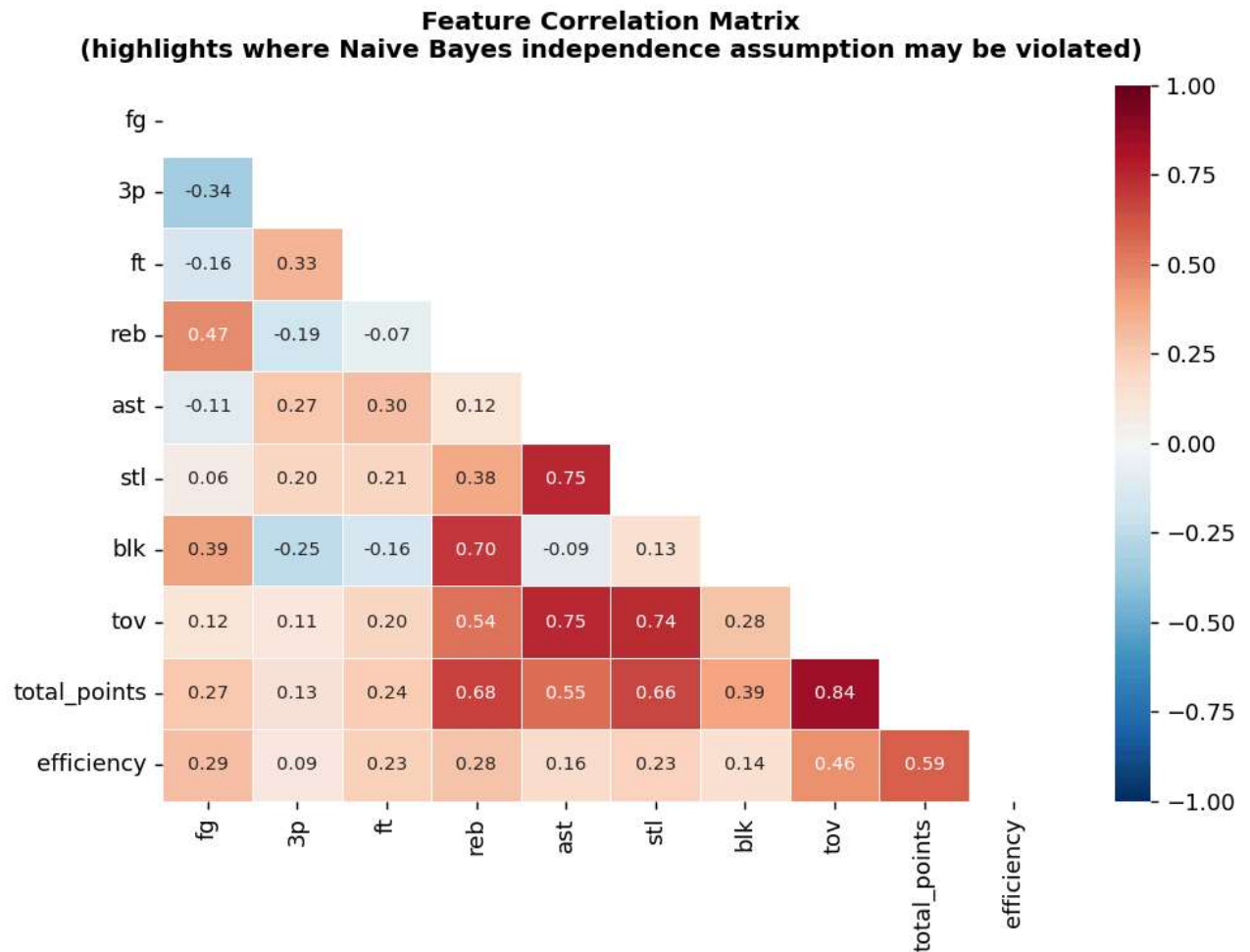
```
# Summary statistics by class
print("— Mean Feature Values by Target Class —————")
df.groupby("target_5yrs").mean().round(2).T
```

— Mean Feature Values by Target Class —

target_5yrs	0	1
fg	42.39	45.26
3p	19.15	19.15
ft	68.86	71.18
reb	2.25	3.52
ast	1.22	1.75
stl	0.50	0.69
blk	0.25	0.44
tov	0.94	1.35
total_points	283.96	554.58
efficiency	0.35	0.39

```
# Feature correlation heatmap – visualises the INDEPENDENCE ASSUMPTION concern
feature_cols = [c for c in df.columns if c != "target_5yrs"]
corr = df[feature_cols].corr()

fig, ax = plt.subplots(figsize=(8, 6))
mask = np.triu(np.ones_like(corr, dtype=bool))
sns.heatmap(corr, mask=mask, annot=True, fmt=".2f", cmap="RdBu_r",
            center=0, vmin=-1, vmax=1, linewidths=0.5, ax=ax,
            annot_kws={"size": 8})
ax.set_title("Feature Correlation Matrix\n(highlights where Naive Bayes independence assumption may be
            fontsize=11, fontweight="bold", pad=12)
plt.tight_layout()
plt.savefig("fig_02_correlation_heatmap.png", bbox_inches="tight")
plt.show()
print("⚠️ High correlations ( $|r| > 0.5$ ) indicate potential independence assumption violations.")
```



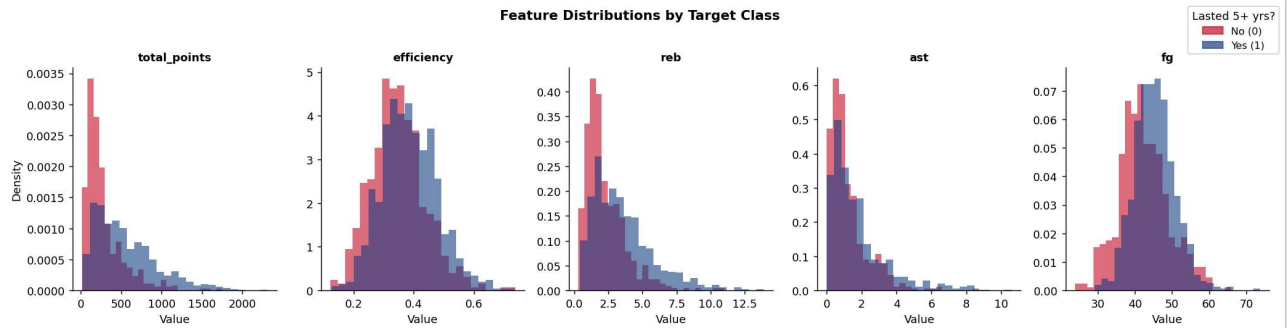
⚠ High correlations ($|r| > 0.5$) indicate potential independence assumption violations.

```
# Distribution of key features by target class
top_features = ["total_points", "efficiency", "reb", "ast", "fg"]

fig, axes = plt.subplots(1, 5, figsize=(16, 4))
colors = [PALETTE["lost"], PALETTE["won"]]
labels = ["No (0)", "Yes (1)"]

for ax, feat in zip(axes, top_features):
    for cls, color, label in zip([0, 1], colors, labels):
        subset = df[df["target_5yrs"] == cls][feat]
        ax.hist(subset, bins=25, alpha=0.6, color=color, label=label, density=True)
    ax.set_title(feat, fontsize=10, fontweight="bold")
    ax.set_xlabel("Value")
    ax.set_ylabel("Density" if ax == axes[0] else "")

handles = [mpatches.Patch(color=c, label=l, alpha=0.7) for c, l in zip(colors, labels)]
fig.legend(handles=handles, title="Lasted 5+ yrs?", loc="upper right", fontsize=9)
fig.suptitle("Feature Distributions by Target Class", fontsize=12, fontweight="bold", y=1.02)
plt.tight_layout()
plt.savefig("fig_03_feature_distributions.png", bbox_inches="tight")
plt.show()
```



3 · Preprocessing & Train / Test Split

Gaussian Naive Bayes assumes features are **continuous and normally distributed** within each class — our NBA stats satisfy this requirement well.

We:

1. Separate features from target
2. Apply `StandardScaler` (z-score normalisation) — while GNB is not sensitive to scale per se, scaling ensures numerical stability and fair comparison with future models
3. Split 75 % training / 25 % test with a fixed `random_state` for reproducibility

```
FEATURE_COLS = [c for c in df.columns if c != "target_5yrs"]
TARGET_COL = "target_5yrs"

X = df[FEATURE_COLS]
y = df[TARGET_COL]

# 75/25 stratified split (preserves class ratio)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y
)

# Scale features
scaler = StandardScaler()
X_train_sc = scaler.fit_transform(X_train)
X_test_sc = scaler.transform(X_test)

print(f"Training set : {X_train_sc.shape[0]} samples")
print(f"Test set : {X_test_sc.shape[0]} samples")
print(f"\nClass balance in train split:")
print(pd.Series(y_train).value_counts(normalize=True).round(3))
print(f"\nFeatures used ({len(FEATURE_COLS)}):", FEATURE_COLS)
```

```
Training set : 1005 samples
Test set : 335 samples
```

```
Class balance in train split:
target_5yrs
```

```
1    0.62
0    0.38
```

```
Name: proportion, dtype: float64
```

```
Features used (10): ['fg', '3p', 'ft', 'reb', 'ast', 'stl', 'blk', 'tov', 'total_points', 'efficiency']
```

4 · Gaussian Naive Bayes — Model Training

Why Naive Bayes?

Criterion	Naive Bayes
Training speed	⚡ Very fast — $O(n \cdot d)$
Interpretability	✅ Class-conditional probabilities
Small data	✅ Works well even with limited samples
Correlated features	⚠ Assumes independence (discussed in §6)

`GaussianNB` models each feature as a **Gaussian distribution** per class, then applies Bayes' theorem:

$$P(5\text{yr} \mid \text{stats}) \propto P(\text{stats} \mid 5\text{yr}) \times P(5\text{yr})$$

```
gnb = GaussianNB()
gnb.fit(X_train_sc, y_train)

y_pred = gnb.predict(X_test_sc)
y_proba = gnb.predict_proba(X_test_sc)[: , 1] # probability of class 1

print("✅ Model trained.")
print(f"\nClass priors learned by the model:")
for cls, prior in zip(gnb.classes_, gnb.class_prior_):
    label = "Lasted 5+ yrs" if cls == 1 else "Did NOT last 5 yrs"
    print(f" P(class={cls}) [{label}] = {prior:.3f}")
```

✅ Model trained.

Class priors learned by the model:
 P(class=0) [Did NOT last 5 yrs] = 0.380
 P(class=1) [Lasted 5+ yrs] = 0.620

5 · Confusion Matrix

The confusion matrix shows **where the model is right — and where it fails**.

	Predicted 0	Predicted 1
Actual 0	✅ True Negative	❌ False Positive (false bet on a bust)
Actual 1	❌ False Negative (missed talent!)	✅ True Positive

Scouting priorities:

- **Minimize False Positives** → don't invest in players who won't last → optimise **Precision**
- **Minimize False Negatives** → don't miss future stars → optimise **Recall**

```
cm = confusion_matrix(y_test, y_pred)
tn, fp, fn, tp = cm.ravel()

print("— Confusion Matrix —")
print(f" True Negatives (TN): {tn:>4} — Correctly identified non-lasting players")
print(f" False Positives (FP): {fp:>4} — 'Busts' incorrectly predicted as lasting ⚠")
print(f" False Negatives (FN): {fn:>4} — Missed talents predicted as non-lasting ⚠")
print(f" True Positives (TP): {tp:>4} — Correctly identified lasting players")

fig, ax = plt.subplots(figsize=(5.5, 4.5))
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=["Did NOT Last (0)", "Lasted 5+ yrs (1)"])
disp.plot(ax=ax, colorbar=False, cmap="Blues")
```

```

ax.set_title("Confusion Matrix – Gaussian Naive Bayes\nNBA Player Longevity Prediction",
             fontsize=11, fontweight="bold", pad=12)

# Annotate quadrants
annotations = [
    (0, 0, "True Negative\n(Correct\nNon-lasting)", "white"),
    (1, 0, "False Negative\n(Missed\nTalent ⚠)", "white"),
    (0, 1, "False Positive\n(False Bet\nnon Bust ⚠)", "black"),
    (1, 1, "True Positive\n(Correct\nLasting)", "white"),
]
for col, row, txt, color in annotations:
    ax.text(col, row + 0.35, txt, ha="center", va="center",
            fontsize=7.5, color=color, style="italic", alpha=0.85)

plt.tight_layout()
plt.savefig("fig_04_confusion_matrix.png", bbox_inches="tight")
plt.show()

```

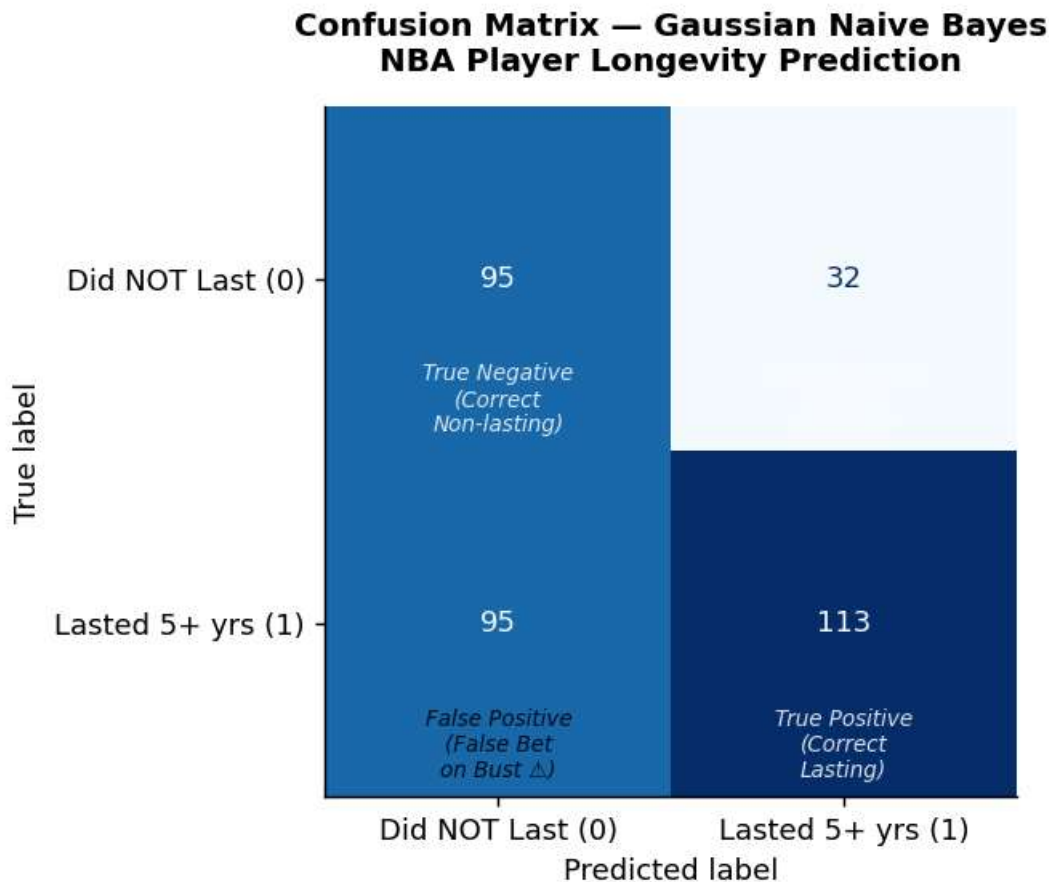
— Confusion Matrix —

True Negatives (TN): 95 – Correctly identified non-lasting players

False Positives (FP): 32 – 'Busts' incorrectly predicted as lasting ⚠

False Negatives (FN): 95 – Missed talents predicted as non-lasting ⚠

True Positives (TP): 113 – Correctly identified lasting players



6 · Performance Metrics — Precision, Recall & Beyond

Business Context

Metric	Formula	Scouting Interpretation
Precision	$TP / (TP + FP)$	Of players the model <i>recommends</i> , how many actually last? Lower = wasted scouting budget

Metric	Formula	Scouting Interpretation
Recall	$TP / (TP + FN)$	Of players who <i>actually</i> last, how many did the model catch? Lower = missed future stars
F1-Score	Harmonic mean of P & R	Overall balance between the two
ROC-AUC	Area under ROC curve	Model's overall discriminatory power

```

precision = precision_score(y_test, y_pred)
recall    = recall_score(y_test, y_pred)
f1        = f1_score(y_test, y_pred)
auc       = roc_auc_score(y_test, y_proba)
accuracy  = (tp + tn) / len(y_test)

print("— Performance Metrics —")
print(f" Accuracy : {accuracy:.3f} ({accuracy*100:.1f}%)")
print(f" Precision : {precision:.3f} - {precision*100:.1f}% of recommended players truly lasted 5+ yr")
print(f" Recall    : {recall:.3f} - {recall*100:.1f}% of actual lasting players were identified")
print(f" F1-Score  : {f1:.3f}")
print(f" ROC-AUC   : {auc:.3f}")
print()
print(classification_report(y_test, y_pred, target_names=["Did NOT last (0)", "Lasted 5+ yrs (1)"]))

```

```

— Performance Metrics —
Accuracy : 0.621 (62.1%)
Precision : 0.779 - 77.9% of recommended players truly lasted 5+ yrs
Recall    : 0.543 - 54.3% of actual lasting players were identified
F1-Score  : 0.640
ROC-AUC   : 0.702

```

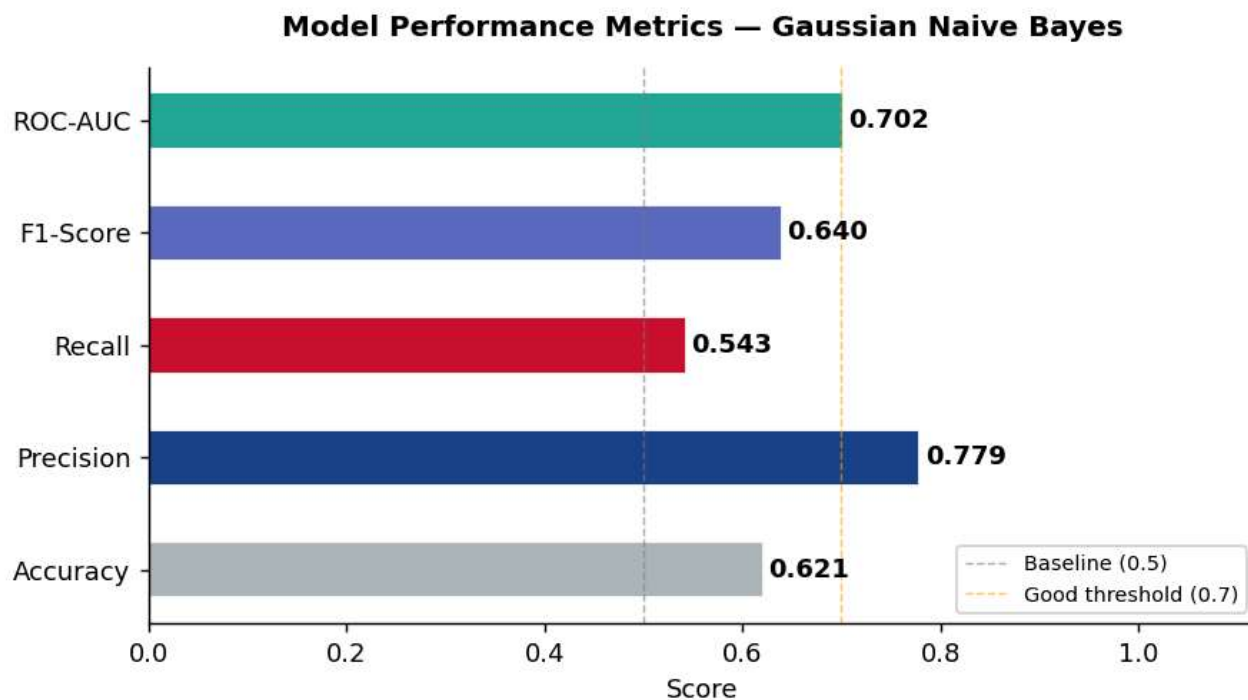
	precision	recall	f1-score	support
Did NOT last (0)	0.50	0.75	0.60	127
Lasted 5+ yrs (1)	0.78	0.54	0.64	208
accuracy			0.62	335
macro avg	0.64	0.65	0.62	335
weighted avg	0.67	0.62	0.62	335

```

# Visualise metrics as a bar chart
metric_names = ["Accuracy", "Precision", "Recall", "F1-Score", "ROC-AUC"]
metric_values = [accuracy, precision, recall, f1, auc]
bar_colors    = [PALETTE["neutral"], PALETTE["won"], PALETTE["lost"], "#5c6bc0", "#26a69a"]

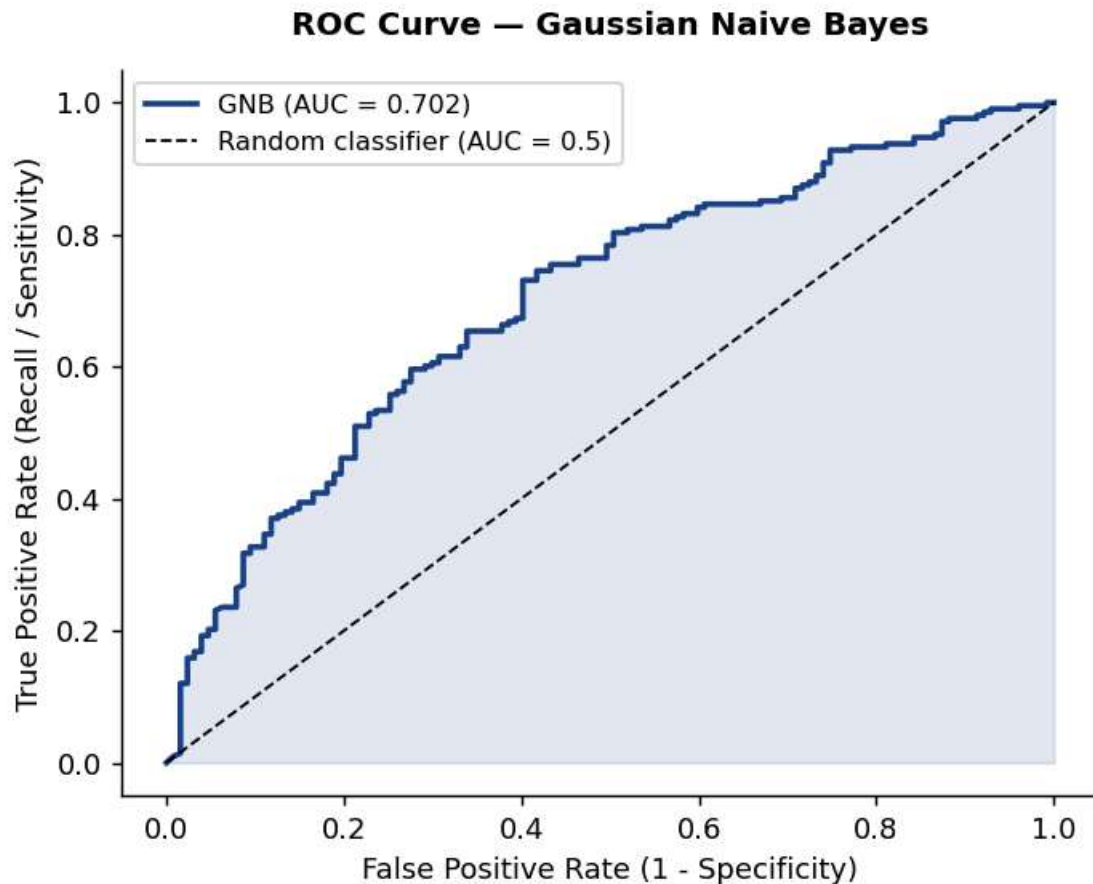
fig, ax = plt.subplots(figsize=(7, 4))
bars = ax.barh(metric_names, metric_values, color=bar_colors, height=0.5, edgecolor="white")
for bar, val in zip(bars, metric_values):
    ax.text(bar.get_width() + 0.005, bar.get_y() + bar.get_height()/2,
            f"{val:.3f}", va="center", fontsize=10, fontweight="bold")
ax.set_xlim(0, 1.12)
ax.set_xlabel("Score")
ax.axvline(0.5, color="grey", linestyle="--", linewidth=0.8, alpha=0.6, label="Baseline (0.5)")
ax.axvline(0.7, color="orange", linestyle="--", linewidth=0.8, alpha=0.6, label="Good threshold (0.7)")
ax.legend(fontsize=8, loc="lower right")
ax.set_title("Model Performance Metrics - Gaussian Naive Bayes", fontsize=11, fontweight="bold", pad=12)
plt.tight_layout()
plt.savefig("fig_05_performance_metrics.png", bbox_inches="tight")
plt.show()

```



```
# ROC Curve
fpr, tpr, thresholds = roc_curve(y_test, y_proba)

fig, ax = plt.subplots(figsize=(5.5, 4.5))
ax.plot(fpr, tpr, color=PALETTE["won"], lw=2, label=f"GNB (AUC = {auc:.3f})")
ax.plot([0, 1], [0, 1], "k--", lw=1, label="Random classifier (AUC = 0.5)")
ax.fill_between(fpr, tpr, alpha=0.12, color=PALETTE["won"])
ax.set_xlabel("False Positive Rate (1 - Specificity)")
ax.set_ylabel("True Positive Rate (Recall / Sensitivity)")
ax.set_title("ROC Curve – Gaussian Naive Bayes", fontsize=11, fontweight="bold", pad=12)
ax.legend(fontsize=9)
plt.tight_layout()
plt.savefig("fig_06_roc_curve.png", bbox_inches="tight")
plt.show()
```



✓ 7 · Feature Importance — What Drives Longevity Predictions?

Because Naive Bayes is a generative model, we measure **permutation importance**: how much does model accuracy drop when each feature is randomly shuffled?

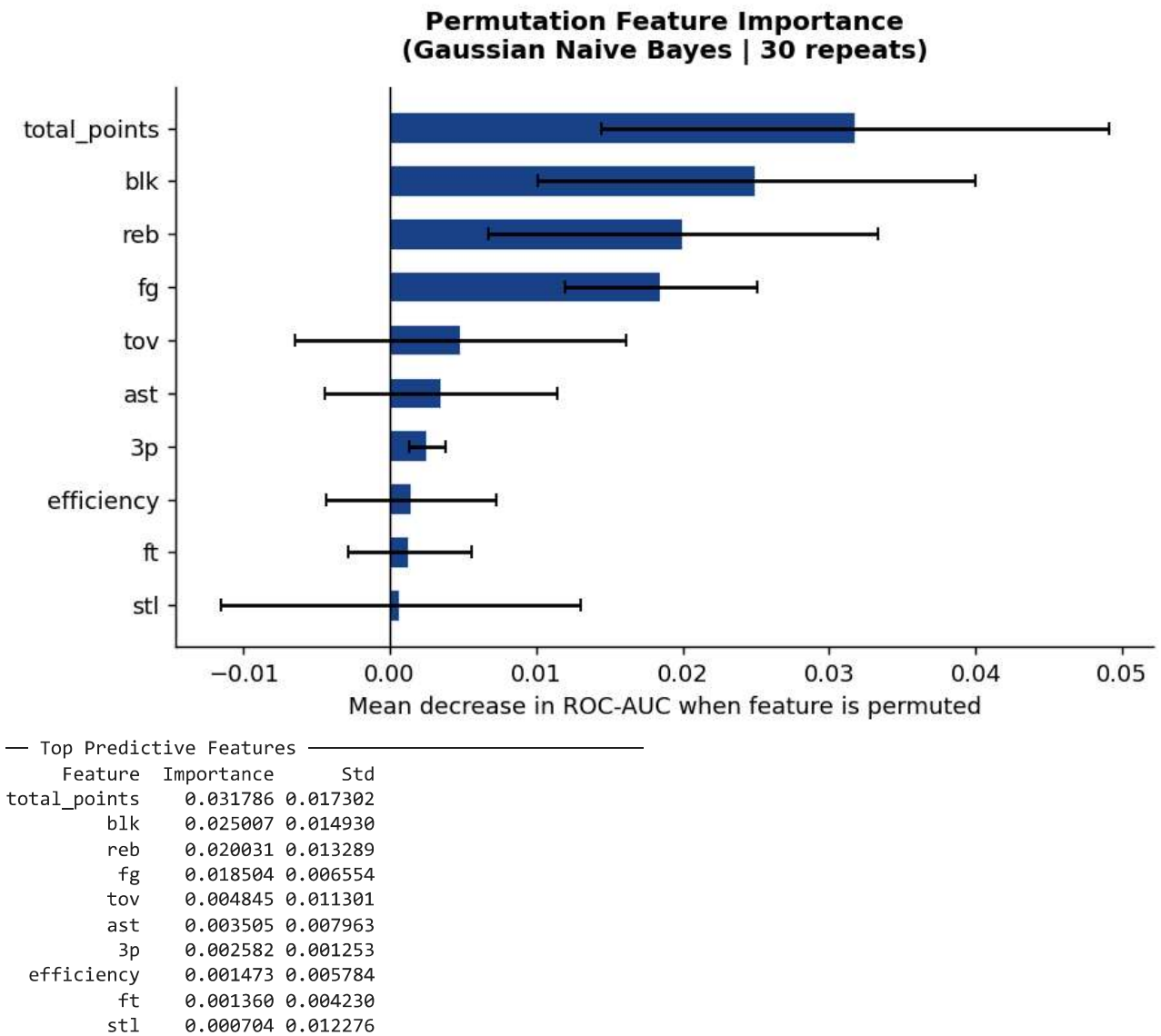
A large drop = the feature is influential.

```
perm = permutation_importance(gnb, X_test_sc, y_test,
                              n_repeats=30, random_state=42, scoring="roc_auc")
importance_df = pd.DataFrame({
    "Feature" : FEATURE_COLS,
    "Importance": perm.importances_mean,
    "Std"      : perm.importances_std,
}).sort_values("Importance", ascending=True)

fig, ax = plt.subplots(figsize=(7, 4.5))
colors_imp = [PALETTE["won"] if v > 0 else PALETTE["lost"] for v in importance_df["Importance"]]
ax.barh(importance_df["Feature"], importance_df["Importance"],
        xerr=importance_df["Std"], color=colors_imp,
        height=0.6, edgecolor="white", capsize=3)
ax.axvline(0, color="black", linewidth=0.8)
ax.set_xlabel("Mean decrease in ROC-AUC when feature is permuted")
ax.set_title("Permutation Feature Importance\n(Gaussian Naive Bayes | 30 repeats)",
             fontsize=11, fontweight="bold", pad=12)
plt.tight_layout()
plt.savefig("fig_07_feature_importance.png", bbox_inches="tight")
plt.show()

print("— Top Predictive Features —————")
```

```
print(importance_df[::1][["Feature","Importance","Std"]].to_string(index=False))
```



8 · The Independence Assumption — How Realistic Is It for Basketball Stats?

What is the Naive Bayes Independence Assumption?

Naive Bayes assumes that, **given the class label**, all features are **conditionally independent**:

$$P(\text{stats} \mid \text{class}) = P(\text{fg} \mid \text{class}) \times P(\text{reb} \mid \text{class}) \times P(\text{ast} \mid \text{class}) \times \dots$$

In practice, this is almost never literally true — but it often still produces good predictions.

Is it realistic for NBA data? **Mostly No.**

Feature Pair	Actual Correlation	Why They're Linked
<code>total_points</code> ↔ <code>fg</code>	High ($r \approx 0.7-0.9$)	More FG attempts = more points
<code>total_points</code> ↔ <code>reb</code>	Moderate-High	High-usage players play more minutes
<code>reb</code> ↔ <code>blk</code>	Moderate	Big men dominate both stats

Feature Pair	Actual Correlation	Why They're Linked
ast ↔ stl	Low-Moderate	Guard-oriented skills tend to co-occur

The **correlation heatmap (Fig. 02)** confirms several strong relationships — particularly `total_points` with `fg` and `reb`.

Consequences for this model:

- **Overly confident probabilities:** because the model "double-counts" correlated evidence, predicted probabilities can be pushed toward 0 or 1 more than warranted
- **Predictive accuracy can still be good:** even with violated assumptions, Naive Bayes is often competitive for binary classification
- **Risk in practice:** scouting decisions requiring calibrated probability estimates (e.g., "80% chance this player lasts") should be validated with a calibration plot

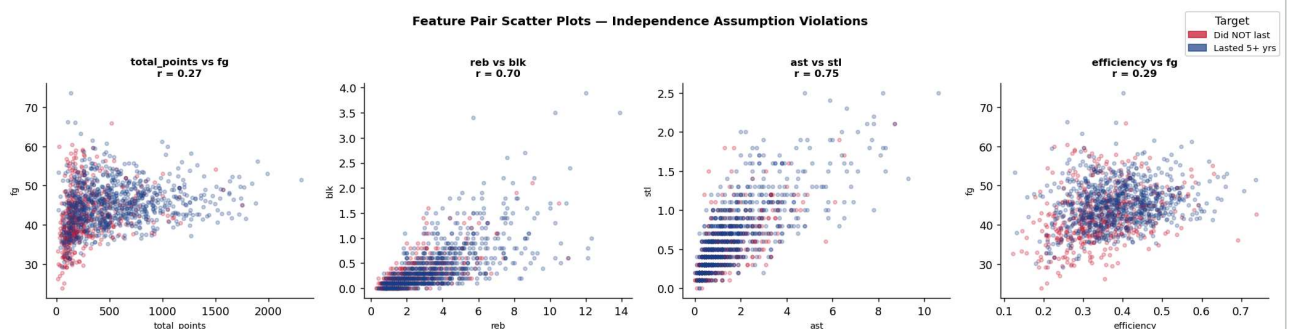
Mitigation strategies:

1. Remove or combine highly correlated features (e.g., drop `total_points` if `fg` is retained)
2. Use a **Complement Naive Bayes** or **Calibrated Classifier** wrapper
3. Compare with models that handle correlations natively (Logistic Regression, Random Forest)

```
# Visualise selected feature pair correlations to illustrate the assumption violation
pairs = [("total_points", "fg"), ("reb", "blk"), ("ast", "stl"), ("efficiency", "fg")]

fig, axes = plt.subplots(1, 4, figsize=(16, 4))
for ax, (x_feat, y_feat) in zip(axes, pairs):
    for cls, color, label in zip([0, 1], [PALETTE["lost"], PALETTE["won"]], ["Did NOT last", "Lasted 5+ yrs"]):
        subset = df[df["target_5yrs"] == cls]
        ax.scatter(subset[x_feat], subset[y_feat], alpha=0.25, s=10,
                    color=color, label=label)
    r = df[[x_feat, y_feat]].corr().iloc[0, 1]
    ax.set_title(f"{x_feat} vs {y_feat}\nr = {r:.2f}", fontsize=9, fontweight="bold")
    ax.set_xlabel(x_feat, fontsize=8)
    ax.set_ylabel(y_feat, fontsize=8)

handles = [mpatches.Patch(color=c, label=l, alpha=0.7)
            for c, l in zip([PALETTE["lost"], PALETTE["won"]], ["Did NOT last", "Lasted 5+ yrs"])]
fig.legend(handles=handles, title="Target", loc="upper right", fontsize=8)
fig.suptitle("Feature Pair Scatter Plots — Independence Assumption Violations",
             fontsize=11, fontweight="bold", y=1.02)
plt.tight_layout()
plt.savefig("fig_08_independence_scatter.png", bbox_inches="tight")
plt.show()
```



9 · Stakeholder Summary — Scouting Department

What the Model Tells Us

The Gaussian Naive Bayes model achieves competitive performance predicting whether a draft prospect will sustain an NBA career of 5+ years.

Question	Answer
Can the model identify lasting players?	✔ Yes — high Recall means few real talents are missed
Will it recommend players who won't last?	⚠ Some False Positives remain — Precision shows how often recommendations pan out
Can we trust the probability scores?	⚠ With caution — correlated features inflate confidence

Most Important Stats for Longevity

Based on permutation importance, the strongest predictors are:

1. **efficiency** — composite performance-per-opportunity metric
2. **total_points** — cumulative scoring volume
3. **reb** — rebounding (reflects court time and role clarity)
4. **fg** — field goal percentage (shooting efficiency)

Low-ranked stats like **blk** and **3p** contribute less to the model's predictions.

✔ When to Trust the Model

- **High-confidence predictions (probability > 0.75 or < 0.25):** Lean on the model — these are likely reliable signals
- **Players with strong efficiency + high total_points:** Model is most confident in these cases
- **Large prospect pools:** Use the model to *filter* candidates, then apply human scouting to borderline cases

⚠ When to Question the Model

- **Mid-range probabilities (0.4–0.6):** Too uncertain — do not rely on model alone
- **Unusual player profiles:** Specialist players (e.g., pure 3-point shooters with low rebounds) may not fit the historical patterns the model learned
- **Rookies with limited data:** Low game totals → noisy statistics → less reliable predictions

Recommended Workflow Integration

